

Response-Decompression / Body-Decode Audit — 2026-07-02

Scope: determine whether the OTHER trackers (rutor, kinozal, nnmclub) and the lava-api-go scraper share the same class of defect that took rutracker down — a client that advertises an encoding it cannot decode (specifically brotli), causing `bodyAsText()` / `body` reads to return raw compressed bytes and every HTML parse to fail silently.

Trigger incident (already root-caused + fixed): `.lava-ci-evidence/sixth-law-incidents/2026-07-02-rutracker-brotli-undecoded-body.json`.

Vocabulary per §11.4.6: statements are facts backed by captured evidence unless explicitly marked UNCONFIRMED: / UNKNOWN:.

0. The defect class, restated

Two independent mechanisms exist, one per stack:

- **Kotlin / OkHttp (Ktor OkHttp engine OR raw OkHttp).** OkHttp only transparently decompresses a Content-Encoding it negotiated ITSELF. When a caller sets Accept-Encoding manually, OkHttp disables its transparent decompression and hands back the RAW bytes. OkHttp's automatic path only ever advertises gzip and only decodes gzip. It does NOT decode br (brotli).
- **Go net/http.** The Transport auto-adds Accept-Encoding: gzip and transparently decompresses gzip ONLY IF (a) the caller did not set Accept-Encoding manually AND (b) `Transport.DisableCompression == false`. Go's stdlib does NOT decode br at all.

So the defect appears when a client advertises br (or any non-gzip encoding it cannot decode) AND the server honours it. The fix is to NOT advertise br (let OkHttp/Go negotiate gzip transparently), unless a real brotli decoder is wired first.

1. Host-side probes (physical evidence)

Egress: Mullvad WireGuard (server no-osl-wg-103, IP 141.98.253.210)
— confirmed via `am.i.mullvad.net/connected` before probing.

Method: one GET per host, guest/public page, body discarded to `/dev/null`, only response headers captured. Two runs per host:

- **bug pattern** — replicating the app's ORIGINAL rutracker header set: Accept-Encoding: gzip, deflate, br, no client-side decode (--compressed NOT used).
- **control** — Accept-Encoding: gzip only, i.e. exactly what OkHttpClient's automatic path sends.

Host (base URL)	Accept-Encoding: gzip, deflate, br → Content-Encoding	Accept-Encoding: gzip (OkHttp path) → Content-Encoding	Server
rutracker.org / forum/index.php (control tracker)	br	gzip	cloudflare
rutor.info /	gzip	gzip	nginx/1.22.1
kinozal.tv /	br	gzip	cloudflare
nnmclub.to /	br	gzip	cloudflare

All eight requests returned HTTP 200 (no Cloudflare challenge from the Mullvad exit at probe time). Facts:

- rutracker.org, kinozal.tv, nnmclub.to (all Cloudflare) **serve brotli when the client advertises br**. This is what broke rutracker.
- All four hosts, including the three Cloudflare ones, **fall back to gzip when only gzip is advertised** — the OkHttpClient/Go transparent path. This proves the fix (do not advertise br) works against the real hosts.
- rutor.info (nginx) chose gzip even when br was offered, so rutor would not have been broken even by the manual-header pattern; but see §2 — rutor's client never advertises br anyway.

2. Per-tracker client-config findings (Kotlin, core/tracker/*)

rutracker — WAS the bug, NOW FIXED (both construction paths)

- Uses **Ktor OkHttp engine** (`HttpClient(OkHttp)`), the only tracker that does.
- `core/tracker/client/src/main/kotlin/lava/tracker/client/di/TrackerClientModule.kt` (provide `RuTrackerHttpClient`, on-device Hilt path) and `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/RuTrackerHttpClientFactory.kt` (create, cloned-provider path) both **previously** set header("Accept-Encoding", "gzip, deflate, br") in `defaultRequest`.
- Current state: the manual header is REMOVED from both paths; each carries an explanatory comment + evidence pointer. OkHttp now adds its own Accept-Encoding: gzip and decompresses transparently. Probe control above confirms `rutracker.org` returns gzip on that path.
- Regression test present: `core/tracker/rutracker/src/test/kotlin/lava/tracker/rutracker/impl/RuTrackerBodyDecompressionRegressionTest.kt`.
- **Action: none beyond shipping the fixed build.**

rutor — NOT affected

- `core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/http/RuTorHttpClient.kt`: raw `okhttp3.OkHttpClient` (NOT Ktor). Requests set only User-Agent. No `defaultRequest`, NO Accept-Encoding, NO brotli anywhere.
- OkHttp therefore advertises only gzip and decodes it transparently. Probe: `rutor.info` returns gzip.
- **Action: none.**

kinozal — NOT affected (but latent trap — see §5)

- `core/tracker/kinozal/src/main/kotlin/lava/tracker/kinozal/http/KinozalHttpClient.kt`: raw `okhttp3.OkHttpClient`. Requests set only User-Agent. No Accept-Encoding. Bodies decoded windows-1251 → UTF-8 via `bodyString()` on `response.body?.bytes()` (OkHttp has already gunzipped the bytes by then).

- Client advertises only gzip; probe control shows kinozal.tv returns gzip on that path. The bug-pattern probe shows kinozal.tv WOULD serve br if br were advertised — but the client never advertises it.
- **Action: none functionally. See §5 for a hardening note (the server is brotli-capable, so re-introducing a manual br header here would reproduce the exact rutracker bug — worse, silently, because kinozal decodes cp1251 so the garbage would look like a charset problem).**

nnmclub — NOT affected (same latent trap as kinozal)

- `core/tracker/nnmclub/src/main/kotlin/lava/tracker/nnmclub/http/NnmclubHttpClient.kt`: `raw okhttp3.OkHttpClient`. Requests set only `User-Agent`. NO `Accept-Encoding`.
- Client advertises only gzip; probe control shows nnmclub.to returns gzip. Bug-pattern probe shows nnmclub.to serves br when br is advertised.
- **Action: none functionally. See §5.**

(adjacent) app → lava-api-go / Ktor-proxy channel — NOT affected

- `core/network/impl/.../NetworkApiRepositoryImpl.kt` `proxyApi()` builds `HttpClient(OkHttp)` with engine `{ preconfigured = client }` and NO manual `Accept-Encoding`. So the GoApi/proxy client advertises only gzip.
 - This matters because lava-api-go has a RESPONSE-side brotli middleware (`internal/server/brotli.go`) that brotli-compresses its own responses ONLY when the client sends `Accept-Encoding: br`, and mDNS advertises compression: `br,gzip` (`internal/discovery/mdns.go`). Since the Android GoApi client never sends br, lava-api-go never brotli-compresses for it, and the app↔goapi channel is safe.
 - **Action: none. UNCONFIRMED:** whether any non-Lava consumer (a browser, `curl --compressed`, a future client) sends `Accept-Encoding: br` to lava-api-go and cannot decode it — browsers and `curl` decode brotli natively, so this is low-risk, but it is not exercised by an Anti-Bluff test today.
-

3. Go API scraper findings (lava-api-go)

rutracker outbound client — NOT affected

- `internal/rutracker/client.go`: the outbound requests set only Cookie and Content-Type headers (lines 272/320/366/368/413/460/462). **No Accept-Encoding is set.**
- The custom `http.Transport` (built in `NewClient`, ~line 147) pins IPv4 and wires the proxy but does **NOT** set `DisableCompression`, so it defaults to false.
- Therefore Go auto-adds `Accept-Encoding: gzip` and transparently decompresses gzip. It never advertises br, so rutracker returns gzip/identity (proven by the \$1 control probe). No brotli reaches the parser.
- **Action: none.**

curated providers (bitsearch, thepiratebay, yts, flaresolverr) — NOT affected

- All build `&http.Client{... Transport: httpx.NewTransport()}`.
- `internal/httpx/proxy.go NewTransport() = http.DefaultTransport.Clone()` with Proxy set only. `DisableCompression` stays false; no `Accept-Encoding` is set on any request.
- Same transparent-gzip guarantee as above. No brotli advertised.
- **Action: none.**

`internal/server/brotli.go` — response side, correct-by-construction

- This is lava-api-go COMPRESSING its OWN responses (server→client), gated on the client sending `Accept-Encoding: br` and on the `BrotliResponseEnabled` config flag. It is NOT an outbound-scraper decode path and is not the defect class.
 - `internal/config/config.go` also exposes `BrotliRequestDecodeEnabled` (default false) — request-body brotli DECODE is off by default. Not a scraper concern.
 - **Action: none.**
-

4. Verdict matrix

Component	Stack	Advertises br?	Can decode br?	Server serves br (probe)?	Affected?
rutracker (Hilt + factory)	Ktor/ OkHttp	yes → no (fixed)	no	yes	was → fixed
rutor	raw OkHttp	no	no	no (nginx picks gzip)	no
kinozal	raw OkHttp	no	no	yes (if asked)	no
nnmclub	raw OkHttp	no	no	yes (if asked)	no
app→goapi/ proxy	Ktor/ OkHttp	no	no	n/a (own service)	no
lava-api-go rutracker client	Go net/ http	no	no (stdlib)	n/a	no
lava-api-go curated clients	Go net/ http	no	no (stdlib)	n/a	no

5. Prioritized action list

1. **rutracker — ALREADY FIXED (P0, done).** Both client-construction paths have the manual Accept-Encoding removed and a regression test. Only remaining step is shipping/redistributing the fixed build. No code change owed.
2. **No functional fix needed for rutor / kinozal / nnmclub / Go API — PROVEN safe** by source audit (no manual Accept-Encoding, Go DisableCompression unset) + the \$1 gzip control probes.
3. **Hardening (P2, latent-trap prevention).** kinozal.tv and nnmclub.to are Cloudflare hosts that DO serve brotli when br is advertised (physical proof in §1). If anyone ever adds a manual Accept-Encoding containing br to KinozalHttpClient / NnmclubHttpClient (or any raw-OkHttp tracker), they reproduce the

exact rutracker bug — and for kinozal the raw brotli bytes would be mis-decoded as cp1251 garbage, masquerading as a charset bug. Recommend: (a) a short comment in each raw-OkHttp client stating "do NOT set Accept-Encoding manually — OkHttp negotiates gzip transparently; a manual br would hand back undecoded brotli", mirroring the rutracker fix comment; and/or (b) a lightweight constitution/lint guard that flags a manual Accept-Encoding containing br in any tracker HTTP client that lacks a brotli decoder.

4. **Prerequisite for any future brotli use (P3).** Nothing in the app or Go API requires brotli today. If perf work later wants to advertise br, a real brotli decoder MUST be wired FIRST (client: an okhttp brotli interceptor / Brotli4j; Go: andybalholm/brotli), before the header is added — never the other way round.
-

6. Evidence provenance / Anti-Bluff notes

- All probes ran host-side over Mullvad against PUBLIC guest pages; response bodies were discarded (-o /dev/null), only headers captured to /run/media/milosvasic/DATA4TB/.build-tmp (outside the repo), then deleted.
- No credentials were used or printed; no authenticated HTML was saved anywhere.
- No source file was edited by this audit; nothing was git added/committed.
- Findings for rutor/kinozal/nmclub/Go are backed by both source inspection AND the live gzip-control probes; the "servers are brotli-capable" claim is backed by the bug-pattern probes. Only the two UNCONFIRMED: items in §2/§3 (non-Lava consumers of lava-api-go's brotli response middleware) are not exercised by a test and are marked as such per §11.4.6.